Università degli Studi di Milano - Bicocca

Dipartimento di Informatica, Sistemistica e Comunicazione

Corso di Laurea in Informatica

Progettazione e sviluppo full stack di una piattaforma web per la gestione dei ristoranti

Relatore: *Professore Gabriele Gianini*

Relazione della prova finale di:

Alessandro Brandino

Matricola 899652

Anno Accademico 2024-2025

***Al team Sync Lab**, per l'aiuto e il supporto offerti durante lo stage, e per avermi dato fiducia fin dal primo giorno.*

***Al mio relatore**, per i consigli offerti durante lo stage e la stesura della relazione finale.*

***Ai miei amici**, per non aver mai dubitato di me. Per le risate, i momenti condivisi e la leggerezza che hanno saputo regalarmi anche nei periodi più complicati.*

***Ai miei compagni di corso**, colonne portanti di questo mio traguardo; per ogni esame insieme, ogni lezione, ogni risata e ogni presa in giro.*

***A tutta la mia famiglia**, che non ha mai dubitato di me e mi ha supportato con affetto festeggiando ogni mio traguardo.*

***Ai nonni**, che ho sempre sentito al mio fianco e che lo saranno anche questa volta.*

***Ai miei genitori**, perché ogni cosa che toccano diventa oro. Per il loro supporto incondizionato di fronte a ogni sfida, ogni mio successo e ogni mio fallimento. Per essere il mio punto fermo, ieri, oggi e domani.*

***A me stesso**, per non aver mollato nei momenti difficili ed essere sempre andato avanti, giorno dopo giorno.*

Sommario

Il presente elaborato descrive la mia esperienza di tirocinio svolta presso l’azienda Sync Lab, durante la quale ho potuto progettare e sviluppare un sistema web full stack per la gestione dei ristoranti. Partendo da un’esigenza reale osservata in prima persona nel contesto della ristorazione, l'obiettivo era quello di creare una piattaforma moderna, intuitiva e accessibile, pensata per rendere più semplice la gestione da parte di proprietari e dipendenti.

L’applicazione è stata sviluppata basandosi su un’architettura a microservizi, con un backend in Spring Boot, un frontend sviluppato in Angular e un database MySql. Il sistema include diverse funzionalità chiave come registrazione/login degli utenti, gestione dei ristoranti, tavoli, menù e prenotazioni, supportando ruoli differenziati (Owner, Admin, Base).

Un’attenzione particolare è stata dedicata alla sicurezza: le API sono protette tramite JWT (JSON Web Token); l’accesso alle funzionalità è regolato da controlli di autorizzazione basati sui ruoli; le password vengono cifrate con l’algoritmo BCrypt, che rende impossibile risalire alla password originale in caso di accesso non autorizzato al database.

Sono state approfondite tecnologie moderne e buone pratiche di sviluppo codice, con attenzione ad una progettazione scalabile. Il progetto ha incluso sia test automatici che validazioni manuali.

Questa esperienza ha rappresentato un’importante opportunità di crescita personale e professionale, permettendomi di entrare in contatto con tecnologie fino ad allora mai utilizzate. Ho acquisito competenze tecniche importanti e una maggiore consapevolezza dei processi tipici di un contesto aziendale reale.

Indice

1 Introduzione....1
1.1 Azienda....1
1.2 Stage....1
1.3 Il progetto....2
1.3.1 L'applicazione....2
1.3.2 I requisiti....2
2 Strumenti e tecnologie....7
2.1 Introduzione....7
2.2 Framework....7
2.2.1 Spring....7
2.2.2 Angular....10
2.2.3 MySql....11
2.3 Ambienti di sviluppo....11
Funzionalità Principali (Requisiti funzionali)....12
3 Progettazione....13
3.1 BackEnd....13
3.1.1 Architettura....13
3.1.2 Gestione chiamate API....13
3.1.2.1 API senza autorizzazioni....14
3.1.2.2 API con autorizzazione....15
3.2 FrontEnd....22
3.2.1 Architettura....22
3.2.2 Struttura....23
3.2.3 Funzionalità....25
4. Test....31
4.1 Test unitari backend....31
4.2 Test umani....32
5. Conclusione....33
5.1 Sviluppi futuri....33
5.2 Conclusione....33
Bibliografia....35

1 Introduzione

1. Azienda

L'azienda presso cui ho svolto lo stage è Sync Lab. Originariamente una Software House, Sync Lab [1] si è espansa diventando un System Integrator grazie all'acquisizione di competenze variegate di tipo tecnologico e applicativo nel campo del software. Oggi è riconosciuta come Innovative Company, focalizzata sulla trasformazione digitale, sviluppando prodotti e soluzioni per vari settori, tra cui GDPR, Big Data, Cloud Computing, IoT, Mobile e Cyber Security. Il suo obiettivo principale è supportare i clienti nella realizzazione, implementazione e gestione di soluzioni IT.

Sync Lab ha realizzato diversi progetti significativi, tra cui:

- **SynClinic**: un software integrato per la gestione dei processi clinici e amministrativi nelle strutture sanitarie.
- **Fast Reservation**: una suite per la prenotazione semplice e affidabile di spazi e servizi per eventi.
- **Seastream**: una piattaforma per migliorare efficienza, sicurezza e innovazione nel settore marittimo.
- **Sobereye:** un software che permette di monitorare la salute e sicurezza sul lavoro verificando lo stato di affaticamento degli occhi.

Sync Lab conta più di 150 clienti diretti, tra cui Unicredit, Poste italiane, Intesa Sanpaolo, Sky, Eni, Enel; e oltre 300 dipendenti distribuiti in 6 sedi in Italia.

1.2 Stage

Sono venuto in contatto con la realtà di Sync Lab tramite il portale dell'università. In seguito a un primo colloquio, mi sono stati presentati gli obiettivi dello stage, elencati di seguito:

- Obiettivi obbligatori:
    - Programmazione java
    - Architettura a microservizi
    - Framework Angular
    - Framework Spring
    - Sviluppo applicazione web utilizzando Spring e Angular
- Obiettivi facoltativi:
    - Approfondimento Spring Security per gestione sicurezza

Prima di avviare l'implementazione del progetto, mi sono concentrato su un ripasso approfondito delle conoscenze acquisite durante il percorso universitario, in particolare relative alla programmazione in Java, alla gestione delle basi di dati e al rispetto dei principi architetturali.

Successivamente, ho dedicato del tempo allo studio degli strumenti principali utilizzati durante il tirocinio, ovvero **Spring** per il backend e **Angular** per il frontend.

1.3 Il progetto

1.3.1 L'applicazione

Avendo lavorato come cameriere per qualche tempo, ho spesso dovuto confrontarmi con gestionali complessi, poco intuitivi e dal design poco curato. Proprio da questa esperienza è nata l'idea di sviluppare un'applicazione gestionale per ristoranti che supporti i proprietari nella gestione del proprio locale, puntando su interfacce **intuitive, moderne e user-friendly**.

Nell'applicazione posso operare 3 diversi ruoli di utente:

1. **Owner**: E' il proprietario del ristorante o dei ristoranti che gestisce.
2. **Admin**: E' un amministratore del ristorante che può fare operazioni invasive al ristorante (creazione/eliminazione di parti importanti)
3. **Base**: E' un dipendente che può visualizzare il ristorante ma può al più modellare le prenotazioni

1.3.2 I requisiti

I requisiti sono divisi in funzionali e non funzionali. Nelle tabelle sottostanti segue la seguente legenda:

- **R** indica un requisito.
- **F**: requisito funzionale, ovvero una funzionalità che deve offrire l'applicazione
- **NF**: requisito non funzionale, che riguarda aspetti come sicurezza, usabilità e performance.
- **Indice**: numero identificativo del requisito

Nella colonna ruolo vengono indicati gli utenti che possono utilizzare la funzionalità corrispondente.

Requisiti	Descrizione	Ruolo
RF1	Registrazione utente con e-mail, nome, cognome, password, ruolo	Tutti
RF1.1	Invio conferma per la registrazione riuscita e reindirizzamento alla pagina del profilo	Tutti
RF1.2	Notifica errore se i dati non sono corretti	
RF2	Login con email e password	Tutti
RF2.1	Reindirizzamento alla pagina del profilo in caso di successo	
RF2.2	Notifica di errore se i dati non sono corretti	
RF3	Creazione ristorante, con nome del ristorante e indirizzo	Owner
RF4	Modifica ristorante	Owner
RF5	Visualizzazione profilo con lista ristoranti	Tutti

Requisiti	Descrizione	Ruolo
RF6	Creazione tavolo	Owner, Admin
RF7	Modifica tavolo	Owner, Admin
RF8	Eliminazione tavolo	Owner, Admin
RF9	Visualizzazione tavoli e prenotazioni	Tutti
RF10	Creazione prenotazione	Tutti
RF11	Gestione prenotazione	Tutti
RF12	Visualizzazione menu	Tutti
RF13	Creazione categoria menu	Owner, Admin
RF14	Creazione piatto menu	Owner, Admin
RF15	Eliminazione categoria menu	Owner, Admin
RF16	Eliminazione piatto menu	Owner, Admin
RF17	Aggiunta dipendente al ristorante	Owner
RNF1	Utilizzabilità dell'applicazione da più dispositivi: desktop, smartphone e tablet	
RNF2	Sicurezza dei dati utente	
RNF3	Sicurezza dei dati del ristorante: Garantire l'accesso a quelle informazione e permettere modifiche solo a chi è autorizzato	
RNF4	Disponibilità dell'applicazione sempre	

1. Requisiti

Descrizione principali casi d'uso

RF1: Registrazione utente
Condizioni: nessuna
Post-condizione: Utente creato e loggato
Requisiti: nome, cognome, email valida, password
Obiettivo: permettere a un nuovo utente di registrarsi come proprietario o dipendente.
Flusso base:

1. Utente apre l'applicazione
    - Sistema: mostra schermata iniziale

2. Utente preme su “Registrati”
    - Sistema: mostra form per registrarti
3. Utente compila e invia il form
    - Sistema: crea l’utente nuovo e lo indirizza alla pagina del profilo

Scenari alternativi:

1. Utente già registrato: il sistema mostra un messaggio d’errore
2. Form non completo o con dati errati: il sistema mostra un messaggio d’errore

Requisiti non funzionali:

1. Sicurezza: la password viene crittografata e resa sicura prima di essere inserita nel database con un sistema asimmetrico.

RF2: Login utente
Condizioni: Utente già registrato precedentemente
Post-condizioni: Utente loggato
Requisiti: email e password
Obiettivo: permettere a un utente già registrato di accedere ai suoi ristoranti
Flusso base:

1. Utente apre l’applicazione:
    - Sistema: mostra schermata iniziale
2. Utente preme su “Login”
    - Sistema: mostra form di login
3. Utente compila e invia il form
    - Sistema: indirizza l’utente alla pagina del profilo

Scenari alternativi:

1. Utente non registrato o password errata: il sistema mostra un messaggio d’errore
2. Form non completo: il sistema mostra un messaggio d’errore

Requisiti non funzionali:

2. Sicurezza: non viene mostrato l’errore “password sbagliata” ma un errore generale per non rilevare che l’email appartiene a un utente registrato.

RF3: Creazione ristorante
Condizioni: Utente loggato e OWNER
Post-condizioni: Ristorante creato
Requisiti: Nome e indirizzo del ristorante
Obiettivo: Permettere a un utente OWNER di creare un ristorante
Flusso base:

Partenza: pagina profilo dell’utente

3. Utente preme su “Aggiungi ristorante”
    a. Sistema: mostra form di aggiunta ristorante
4. Utente compila e invia il form
    a. Sistema: crea il ristorante

Scenari alternativi:

Form non completo: il sistema mostra un messaggio d'errore

RF4: Modifica ristorante
Condizioni: Utente loggato e OWNER. L'utente deve essere collegato al ristorante che vuole modificare.
Post-condizioni: Ristorante modificato
Requisiti: nuovo nome e/o nuovo indirizzo
Obiettivo: Permettere a un utente OWNER di modificare un ristorante
Flusso base:
Partenza: pagina profilo dell'utente

1. Utente preme su "Modifica" accanto al ristorante da modificare
    a. Sistema: mostra form di modifica ristorante
2. Utente compila e invia il form
    a. Sistema: modifica il ristorante

Scenari alternativi:
1. Form non completo: il sistema mostra un messaggio d'errore

RF6: Creazione tavolo
Condizioni: Utente loggato e con permessi OWNER. L'utente deve essere collegato al ristorante del quale vuole creare il tavolo
Post-condizioni: Tavolo creato
Requisiti: Numero di posti, numero del tavolo (facoltativo), nome del tavolo (facoltativo)
Obiettivo: Permettere a un proprietario di creare il tavolo che sarà poi disponibile.
Flusso base:
Partenza: pagina tavoli del ristorante
1. Utente preme il pulsante "+" che permette l'aggiunta di un tavolo
    - Sistema: mostra form di aggiunta tavolo
2. Utente compila e invia il form
    - Sistema: crea il tavolo. Se il numero del tavolo non è indicato lo genererà automaticamente prendendo il numero più piccolo disponibile

Scenari alternativi:
1. Numero posti non indicato: mostra messaggio d'errore
2. Numero di tavolo già presente: mostra messaggio d'errore

2 Strumenti e tecnologie

2.1 Introduzione

In questo capitolo verranno presentati nel dettaglio gli **strumenti**, gli **ambienti di sviluppo** e le **tecnologie** adottate per la realizzazione dell'applicazione. La scelta di ciascuna tecnologia è stata effettuata prima di tutto in base agli obiettivi del progetto e in base a criteri di efficienza, affidabilità, scalabilità, sicurezza.

2.2 Framework

Per il tirocinio sono stati scelti due framework principali: **Spring** per il backend e **Angular** per il frontend

2.2.1 Spring

Spring [2] è un framework open source che fornisce un approccio modulare per la creazione di applicazioni aziendali Java. La funzionalità principale di Spring è la Dependency Injection (DI), una tecnica di programmazione usata per implementare IoC.

Spring Boot è un modulo che permette di creare applicazioni Spring autonome, configurando automaticamente Spring e altre librerie.

In seguito sono elencati tutti i moduli utilizzati in Spring per l'applicazione.

Spring Data JPA

Spring Data JPA [3] fornisce il supporto di repository per Jakarta Persistence API (JPA), facilitando lo sviluppo di applicazioni che necessitano di accedere a dati JPA. L'interfaccia centrale dell'astrazione del repository Spring Data è Repository [4]. Questa interfaccia funge principalmente da interfaccia indicatore per acquisire i tipi con cui lavorare. Le interfacce CrudRepository[5] e ListCrudRepository[6] forniscono funzionalità CRUD per la classe di entità gestita.

In seguito i metodi principali nativi utilizzabili:

- save (S entity) Salva l'entità
- findById(ID id) Restituisce l'entità identificata dall'id
- existsById(Id id) Indica se esiste l'entità identificata dall'id
- findAll() Restituisce tutte le entità
- count() Conta le entità
- delete (S entity) Cancella l'entità

```java
public interface RestaurantRepo extends
                JpaRepository<Restaurant, Integer> {

    @Override
    Optional<Restaurant> findById(Integer id);
    // Trova un ristorante per id

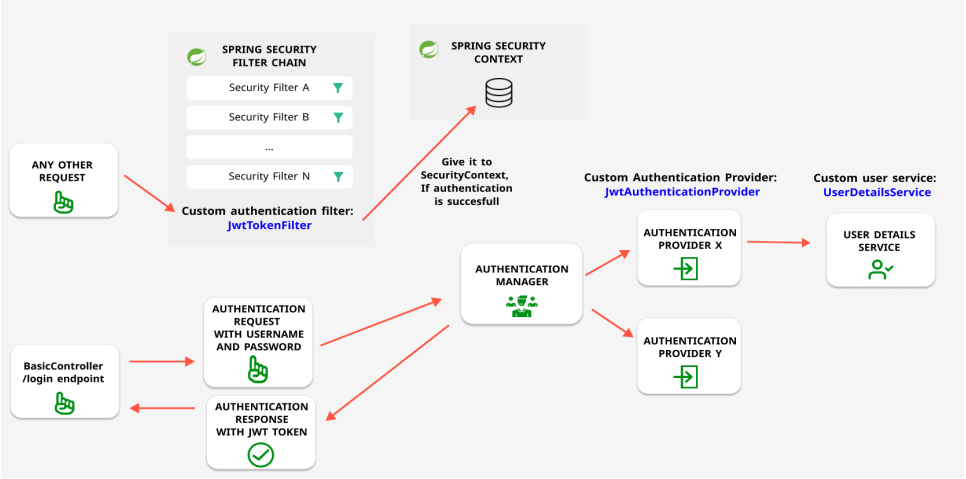
    @Override
    boolean existsById(Integer id);
    // Controlla che il ristorante con id esista

     @Override
    void deleteById(Integer id);
    // Cancella il ristorante con id
}
```

Es. di codice nel quale si utilizza **Spring Data JPA**

Spring Security

Spring Security [7] fornisce strumenti per la gestione della **sicurezza** dell'applicazione, agendo come livello di protezione intermedio tra l'esterno e il backend. In particolare intercetta tutte le richieste HTTP e applica meccanismi di **autenticazione** e **autorizzazione**.



Nella mia applicazione, nello specifico, viene utilizzato per limitare l'accesso delle API solo agli utenti autorizzati seguendo questi criteri:

- **Accesso libero:** Le funzionalità di login e registrazione possono essere raggiunte da chiunque e non richiedono autorizzazione
- **Autorizzazione basata su ruolo:** L'utente deve avere un'adeguata autorizzazione per fare alcune operazioni (ADMIN o OWNER)
- **Associazione a risorse:** L'utente può fare richieste riguardanti solo i ristoranti a cui è collegato

Per fare ciò, a seguito di una registrazione o login da parte di un utente, viene creato un **JWT (Json Web Token)** [8] univoco, inviato poi come risposta a chi ha effettuato l'operazione. Questo token, **firmato digitalmente**, contiene informazioni utili per identificare e autorizzare l'utente.
Il token è formato da tre parti:

- **Header:** è la prima parte del token. Contiene due voci. "typ" che ha come valore "JWT" e "alg" che conterrà l'algoritmo di codifica. Nel mio caso l'algoritmo è HS256.
- **Payload:** è il corpo del JWT e contiene le informazioni (claims) relative all'utente. Nel mio caso specifico, il payload include:
  - sub (subject): rappresenta l'identificativo univoco dell'utente, che in questo caso corrisponde alla email.
  - "iat" (issued at): indica l'istante in cui il token è stato generato in formato timestamp UNIX (secondi).
  - "exp" (expiration): indica l'istante in cui il token scade in formato timestamp UNIX (24 ore dopo la generazione nel mio caso).
- **Signature:** è l'ultima parte del token ed è la firma digitale. La sua funzione è quella di garantire l'integrità e l'autenticità del token. Nel mio caso è calcolata usando l'algoritmo HMAC-SHA256. Questi sono i passaggi:
  - Viene calcolata la codifica in **Base64Url** dell'header e del payload.
  - Le due stringhe ottenute vengono concatenate con un punto in mezzo.
  - Su questa stringa viene applicato l'algoritmo HMAC-SHA256 utilizzando una chiave segreta conosciuta solo dal server.

  Così facendo qualsiasi alterazione dell'header o del payload invaliderebbe il token, in quanto la signature non corrisponderebbe

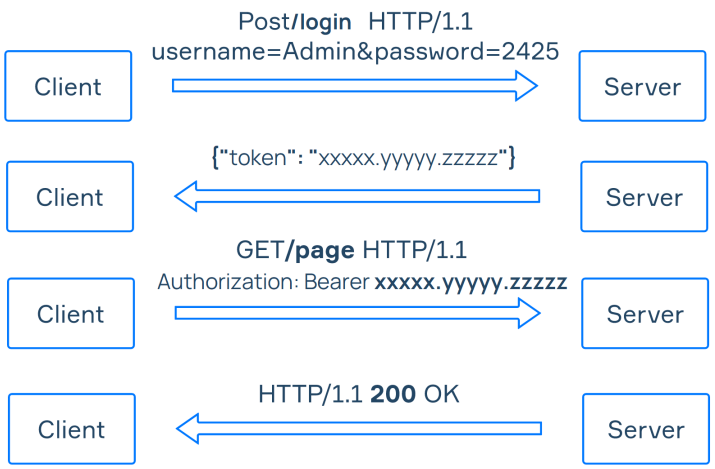
```
{
  "header": {
    "alg": "HS256",
    "typ": "JWT"
  },

  "payload": {
    "sub": "utente@example.com",
    "iat": 1744797600, //Data: 16/04/2025 10:00:00 UTC
    "exp": 1744884000  //Data: 17/04/2025 10:00:00 UTC
  },

  "signature": "d3aL9v4QxkFJ92h_MockSignatureExample_ABC123"
}
```

es JWT

Per accedere alle risorse protette, il client dovrà includere il token nell'**header HTTP Authorization.**



Per garantire un elevato livello di sicurezza, le password degli utenti non vengono salvate nel database in chiaro, ma vengono crittografate tramite il **PasswordEncoder** [9] di Spring Security.
Viene infatti applicata alla password una funzione di hash sicura (nella mia applicazione BCrypt) che rende impossibile risalire alla password originale anche in caso di accesso non autorizzato al database. In fase di login, la password dell'utente utilizza la stessa funzione per poi essere confrontata con la password salvata nel database.

Spring Email

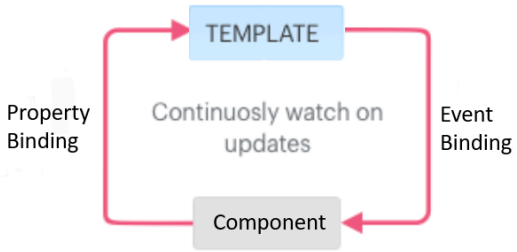
Spring Email[10] è un modulo per l'invio di email tramite SMTP. Viene utilizzato nell'applicazione per mandare agli utenti la prenotazione effettuata.

2.2.2 Angular

Angular [11] è un *framework* per la creazione di **SPA** (Single Page Application) usando **html** e **TypeScript**. Si basa sui component, i quali definiscono le classi che contengono i dati è la logica dell'applicazione; ognuno di essi è associato a un template HTML.
Angular, a differenza di molti altri framework, supporta il **binding bidirezionale** dei dati; quindi i cambiamenti nel DOM e le scelte dell'utente si riflettono sui dati del programma (e viceversa). Questo tipo di binding semplifica la comunicazione tra template e relativo component e anche tra un component padre e un component figlio.

Esistono due tipi di binding:

- Event binding: consente all'applicazione di rispondere all'input dell'utente aggiornando i dati dell'applicazione
- Property binding: consente di inserire nell'HTML dati calcolati dall'applicazione



Two-Way Data Binding

Un *service* è una classe annotata con il decoratore `@Injectable()`, progettata per svolgere uno scopo ben preciso, come ad esempio gestire la comunicazione con un'API, elaborare dati o condividere informazioni tra componenti.

I service promuovono la **separazione delle responsabilità**, in quanto isolano la logica applicativa dal codice di presentazione (i componenti).

Grazie al meccanismo di **Dependency Injection (DI)** fornito da Angular, i componenti possono accedere facilmente ai service: Angular si occupa automaticamente della creazione e del ciclo di vita degli oggetti da iniettare.

Ad esempio, nella mia applicazione uso delle service come tramite per mandare richieste http verso il server. Così che possa utilizzarle in più component senza riscrivere le chiamate API.

2.2.3 MySql

MySQL [12] è uno dei database relazionali open source più usati al mondo. In un contesto di sviluppo web, MySQL viene spesso scelto per memorizzare e gestire i dati strutturati dell'applicazione, come utenti, ordini, prodotti o, nel mio caso, ristoranti e relative informazioni gestionali.
Il database viene organizzato in tabelle, nei quali ogni record ha campi fissati alla creazione dello stesso (o in seguito a una modifica di solito invasiva).

2.3 Ambienti di sviluppo

Per lo sviluppo software, sono stati scelti i seguenti IDE:

- IntelliJ IDEA [13] è stato scelto per le sue funzionalità avanzate che aiutano nell'implementazione, come il refactoring automatizzato e il supporto per molti framework. E' utile soprattutto grazie al completamento automatico del codice che velocizza la produzione del codice. E' stato utilizzato per lo sviluppo backend con Spring
- Visual Studio Code [14] è stato scelto per la sua semplicità e versatilità. Include un debugger per Typescript. Ha un terminale integrato che permette di eseguire comandi

senza usare l’editor. Inoltre ha un integrazione con GIT che facilita le commit del progetto. E’ stato usando Visual Studio Code per lo sviluppo frontend con Angular

- XAMPP [15], è stato scelto perché facile da installare e offre un ambiente completo in locale. Ti permette di gestire mysql via interfaccia web, senza dover usare la riga di comando. L’ho utilizzato per il database mysql.

Funzionalità Principali (Requisiti funzionali)

1. **Registrazione e Login**: Gli utenti possono registrarsi e loggarsi per gestire il proprio ristorante comodamente sull’applicazione web.
2. **Gestione dei Tavoli**: Consente la visualizzazione, l'aggiunta e la modifica dei tavoli del ristorante.
3. **Prenotazioni**: Permette al ristorante di aggiungere prenotazioni per i tavoli, visualizzare le prenotazioni esistenti e gestire lo stato delle prenotazioni.

3 Progettazione

3.1 BackEnd

3.1.1 Architettura

L'applicazione adotta un'architettura a livelli che garantisce separazione delle responsabilità e migliora la manutenibilità del codice:

- **Controller**: Ogni controller è annotato con `@RestController`, che combina le annotazioni `@Controller` e `@ResponseBody`. Le classi controller si occupano di gestire le richieste HTTP in ingresso e restituire i dati al client in formato JSON.
- **Service**: Logica di business e interazioni con il repository. Le classi service sono annotate con `@Service` e vengono registrate come Bean da Spring, permettendo l'iniezione delle dipendenze nei componenti che ne hanno bisogno.
- **Repository**: Accesso ai dati e operazioni CRUD. Le interfacce sono estensioni di JpaRepository e permettono di interagire in modo semplice col database.

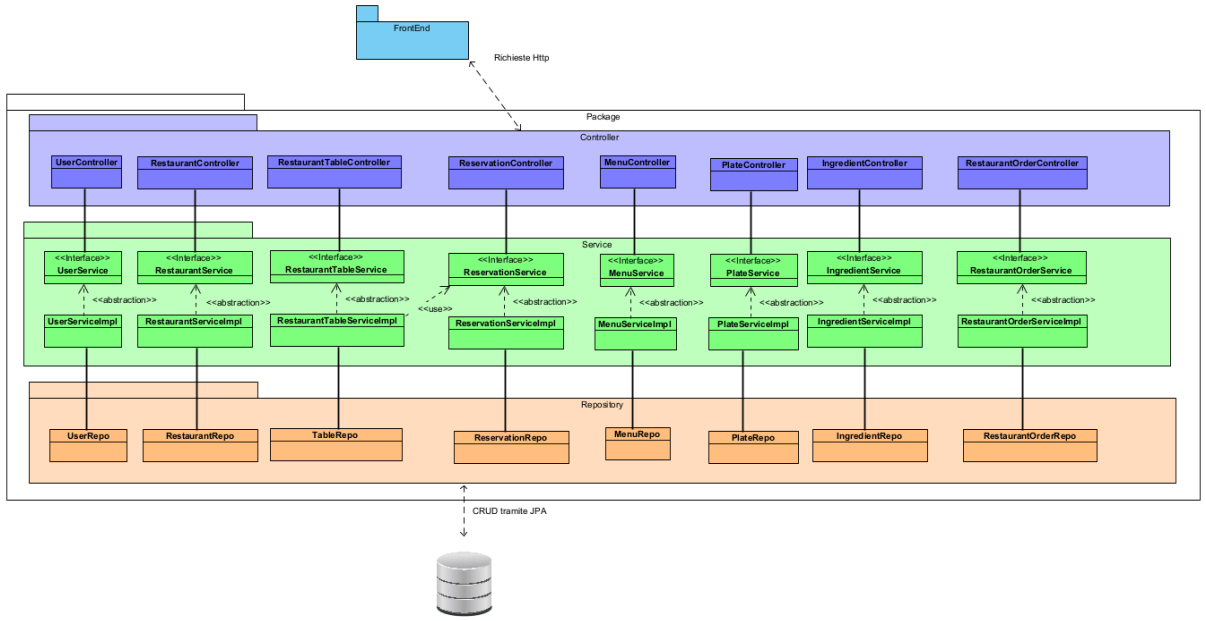


Immagine schema architettura

3.1.2 Gestione chiamate API

In questa sezione verrà analizzato il meccanismo con cui il server gestisce le chiamate provenienti dall'esterno, illustrando il flusso che le chiamate percorrono attraverso i vari componenti del backend. Verranno anche illustrati nello specifico gli aspetti legali alla sicurezza, tra cui autenticazione e autorizzazione.

3.1.2.1 API senza autorizzazioni

Registrazione e Login utente

Durante la fase di registrazione e autenticazione (login) non è richiesto alcun token e non viene fatto alcun controllo. Gli endpoint sono configurati appositamente per bypassare i filtri di sicurezza, così che qualsiasi utente possa fare richiesta per registrarsi o eseguire il login. Una volta fatta l'autenticazione, il server restituirà l'oggetto utente con tutti i suoi dati (tranne la password) con in aggiunta il JWT generato.

```
.authorizeHttpRequests(auth -> auth
  .requestMatchers("/api/users/register",
                   "/api/users/login")
                   .permitAll()
  .anyRequest().authenticated()
)
```

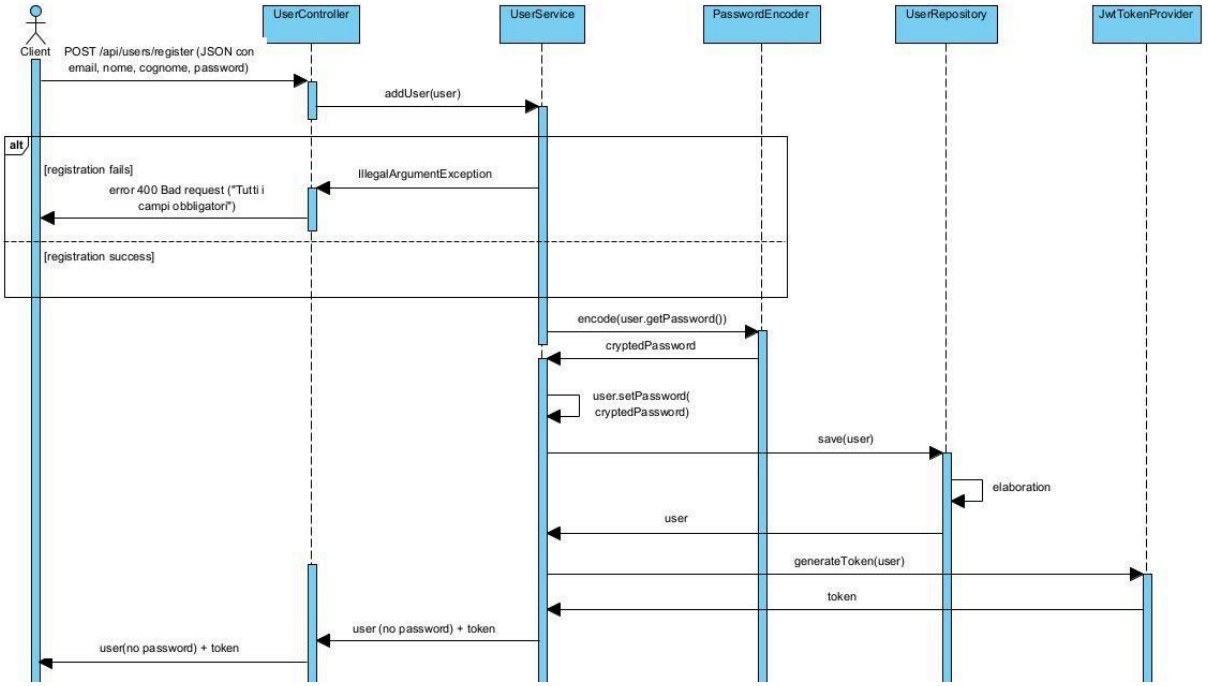


Diagramma di sequenza registrazione

Il processo di registrazione, viene avviato dal client effettuando una richiesta *POST* verso l'endpoint */api/users/register* inviando un oggetto JSON contenente i dati dell'utente (email, nome, cognome, password). l'**UserController** è la classe che riceve la richiesta. Seguendo la filosofia di divisione delle responsabilità, il controller si limiterà a inoltrare la richiesta al **UserService,** il quale si occuperà della logica di business. Il servizio controllerà innanzitutto la presenza di tutti i dati, e, se i dati sono validi procederà a salvare l'utente nel seguente modo:

- La password viene cifrata tramite il componente **PasswordEncoder**
- L'oggetto viene salvato (con la password cifrata) nel database tramite **UserRepository**
- Viene creato il JWT dell'utente tramite il componente **JWTokenProvider**
- Invia al controller come risposta l'utente senza la password e con il token

Alla fine l'**UserController** invia l'utente al client.

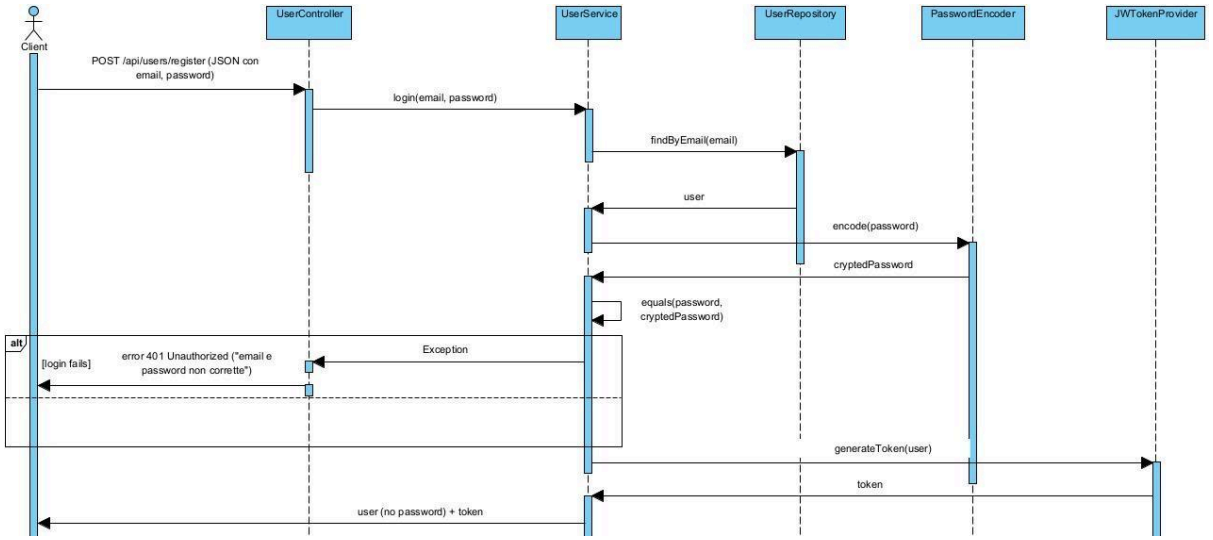


Diagramma di sequenza login

Il processo di autenticazione (login) viene avviato dal client effettuando una richiesta *POST* verso l'endpoint */api/users/login* inviando un oggetto JSON che contiene l'e-mail e la password. come nella registrazione, **UserController** riceverà la richiesta dal client e la inoltrerà a **UserService.** Il servizio esegue i seguenti passaggi:

- Recupera l'utente usando l'email da **UserRepository**
- Crittografa la password passata dal client usando **PasswordEncoder**
- Controlla che la password crittografata sia uguale alla password salvata su database
- Viene creato il JWT dell'utente tramite il componente **JWTokenProvider**
- Invio al controller l'utente senza password e col token

Alla fine l'**UserController** invia l'utente al client.

3.1.2.2 API con autorizzazione

Vedremo come funzionano i controlli di sicurezza per chiamate API protette. Prendiamo ad esempio l'aggiornamento dei dati di un ristorante (nome e/o descrizione). In questo caso, a differenza di login e registrazione, il sistema esegue una serie di operazioni per garantire sicurezza, integrità e controllo dei privilegi.

Aggiornamento Ristorante

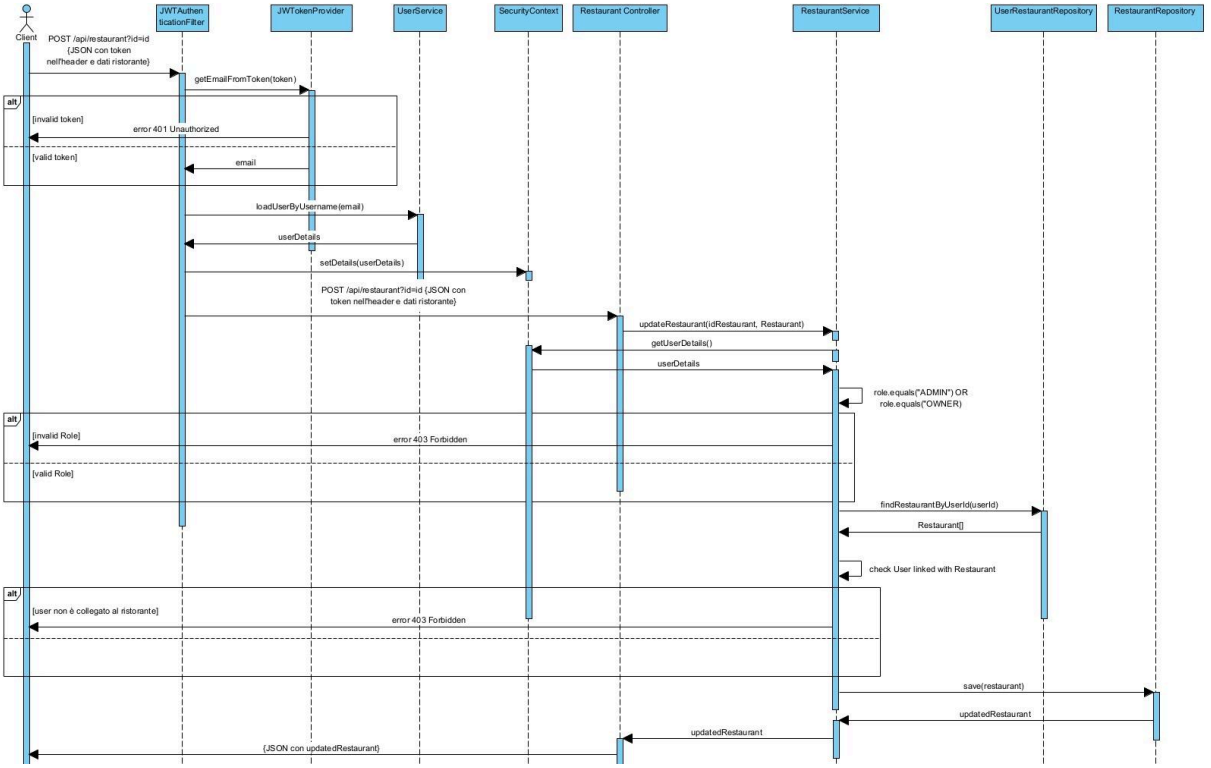


Diagramma di sequenza aggiornamento ristorante

Il processo di aggiornamento del ristorante, avviene con una richiesta *POST* con endpoin /api/restaurants?id={id} contenente:

- **JWT** nell'header
- Oggetto **JSON** con i nuovi dati del ristorante

La richiesta viene intercettata dal **JwtAuthenticationFilter,** che, come dice il nome, fa da filtro tra le chiamate esterne ed il backend. Il filtro ha il compito di:

- Estrarre il token dall'header e validarlo tramite il **JWTokenProvider**
- Se valido, estrarre l'email relativo al token e caricare i dettagli utenti tramite il service **UserService**
- Viene creato un oggetto **Authentication** che viene iniettato nel contesto di sicurezza (nel componente **SecurityContextHolder)**

```
SecurityContextHolder.getContext().setAuthentication(authentication);
```

- Se il token è valido e ha iniettato correttamente l'oggetto **Authentication,** inoltrerà la chiamata *POST* al **RestaurantController**

Il **RestaurantController** verificherà che il ruolo dell'utente è *"ADMIN"* o *"OWNER"* (tramite un metodo della classe **SecurityConfig** non rappresentata). Se l'utente ha la giusta autorizzazione inoltrerà la richiesta a **RestaurantService.**

RestaurantService controllerà il collegamento tra utente e ristorante (solo gli utenti del ristoranti possono modificarlo) e, se tutto corretto, procederà alla modifica del ristorante.

Aggiunta ristorante

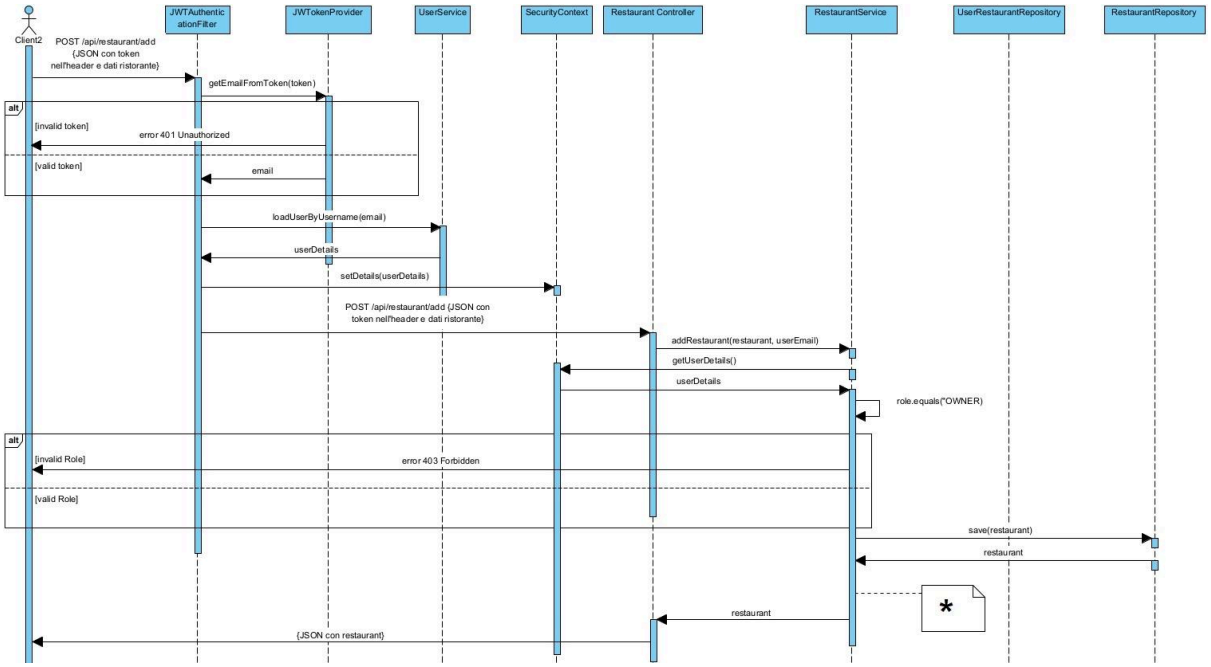


Diagramma di sequenza Aggiunta ristorante

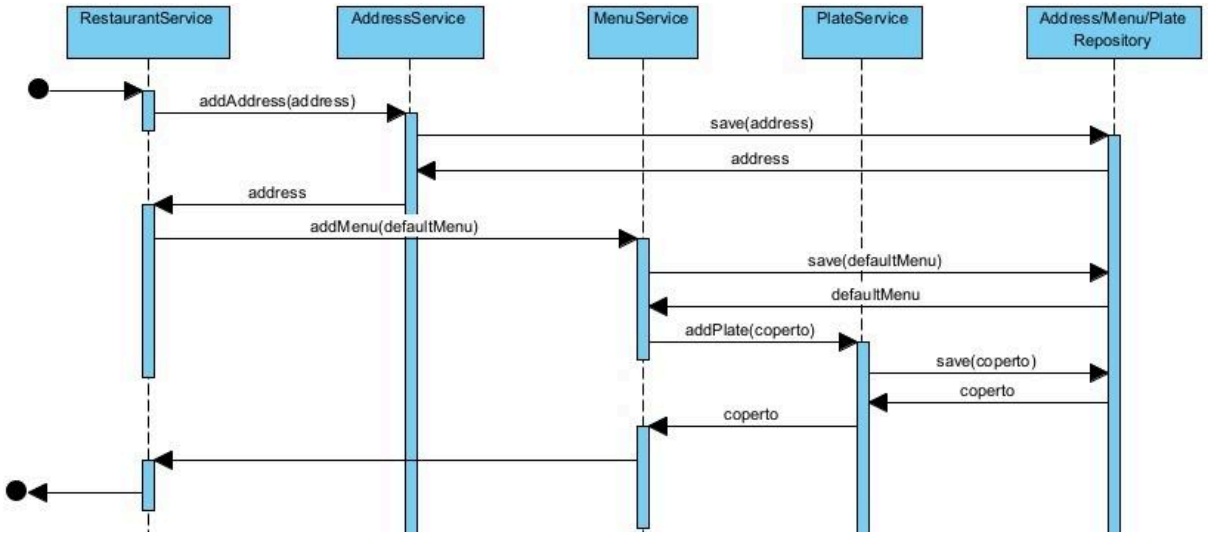


Diagramma di sequenza aggiunta ristorante al punto dell'asterisco

Il processo di aggiunta del ristorante, avviene con una richiesta *POST* con endpoin /api/restaurants/add contenente:

- **JWT** nell'header
- Oggetto **JSON** con i nuovi dati del ristorante (nome, descrizione e indirizzo)

La richiesta viene intercettata dal **JwtAuthenticationFilter** che filtra la richiesta.

Il **RestaurantController** verificherà che il ruolo dell'utente è *"OWNER"* (tramite un metodo della classe **SecurityConfig** non rappresentata). Se l'utente ha la giusta autorizzazione inoltrerà la richiesta a **RestaurantService.**

RestaurantService, verificata la correttezza di tutti i dati, inoltrerà al servizio **AddressService** l'indirizzo da aggiungere per poi aggiungere il ristorante tramite il repository.

RestaurantService fa richiesta a **MenuService** che creerà un Menu vuoto nel quale poi, inoltrando la richiesta a **PlateService,** verrà creato di default il coperto a prezzo 0.

Aggiunta menu

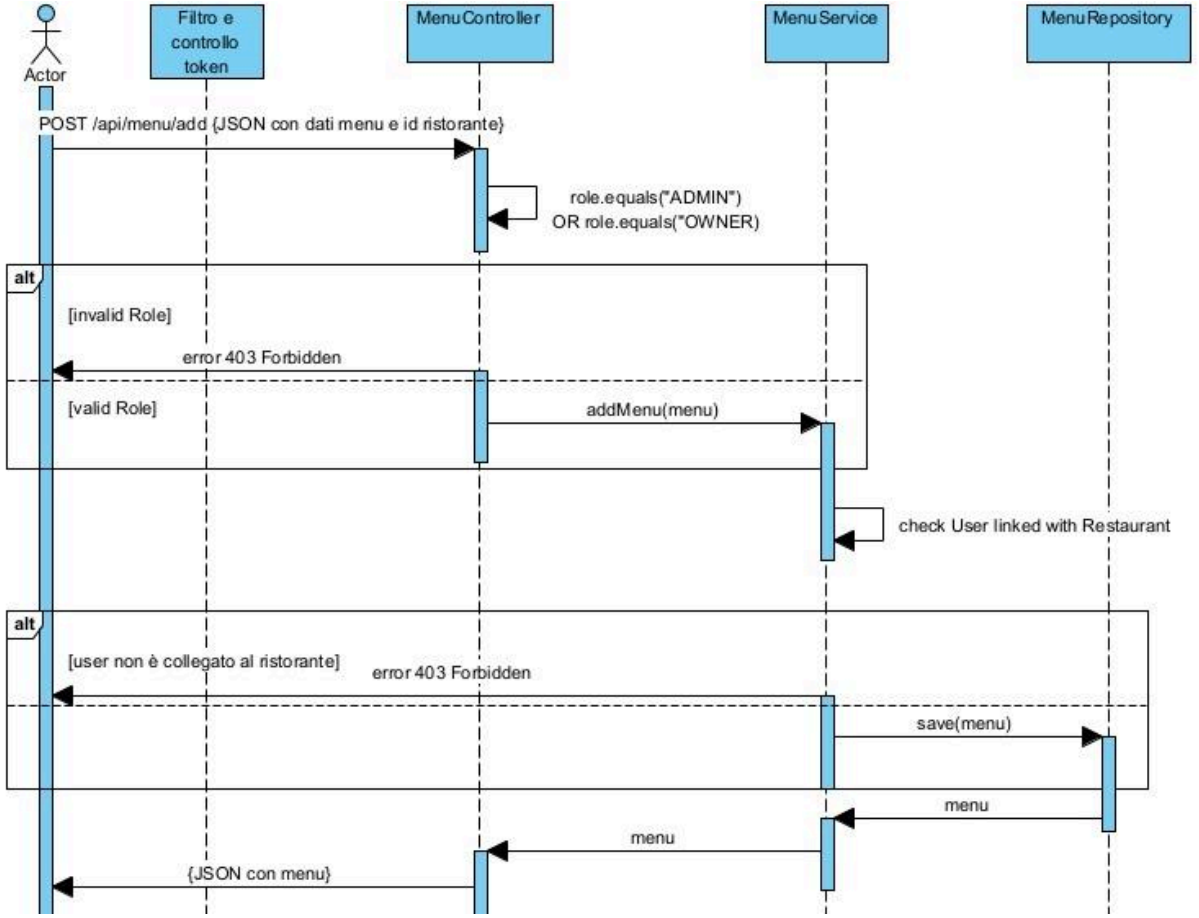


Diagramma di sequenza aggiunto menu

Il processo di aggiunta del menu, avviene con una richiesta *POST* con endpoin /api/menu/add contenente:

- **JWT** nell'header
- Oggetto **JSON** con i dati del menù (nome, descrizione e id del ristorante al quale apparterrà)

La richiesta viene intercettata dal **JwtAuthenticationFilter** che filtra la richiesta (semplificato nel diagramma).

Il **MenuController** verificherà che il ruolo dell'utente è *"OWNER"* o *"ADMIN"* (tramite un metodo della classe **SecurityConfig** non rappresentata). Se l'utente ha la giusta autorizzazione inoltrerà la richiesta a **MenuService.**

MenuService, verificata la correttezza di tutti i dati e verificherà che l'utente sia collegato col ristorante. Infine salverà il menu tramite il repository.

Aggiunta piatto

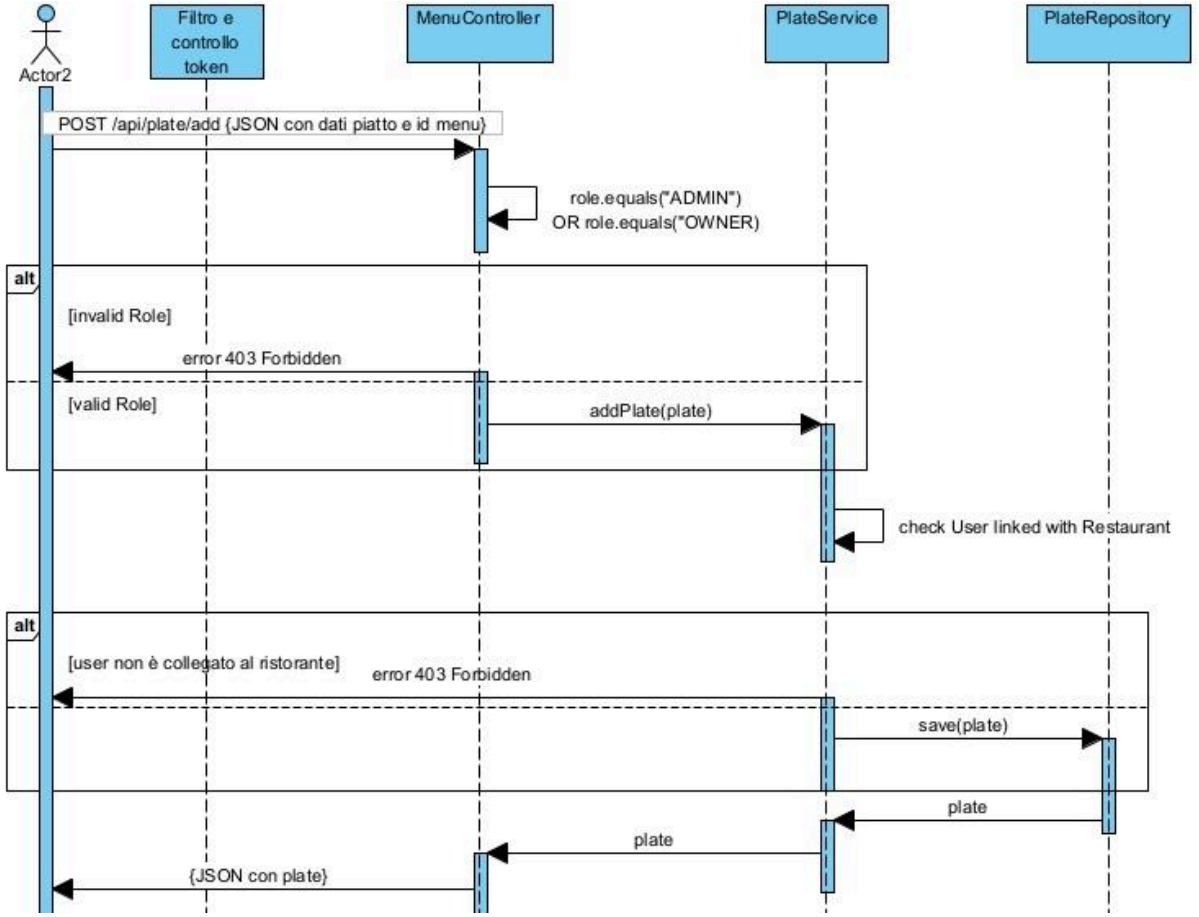


Diagramma di sequenza Aggiunta Piatto

Il processo di aggiunta del piatto, avviene con una richiesta *POST* con endpoin /api/plate/add contenente:

- **JWT** nell'header
- Oggetto **JSON** con i dati del piatto (nome, descrizione, prezzo e id del menu al quale apparterrà)

La richiesta viene intercettata dal **JwtAuthenticationFilter** che filtra la richiesta (semplificato nel diagramma).

Il **PlateController** verificherà che il ruolo dell'utente è *"OWNER"* o *"ADMIN"* (tramite un metodo della classe **SecurityConfig** non rappresentata). Se l'utente ha la giusta autorizzazione inoltrerà la richiesta a **PlateService.**

PlateService, verificata la correttezza di tutti i dati e l'esistenza del menu, verificherà che l'utente sia collegato col ristorante. Infine salverà il menu tramite il repository.

Aggiunta prenotazione

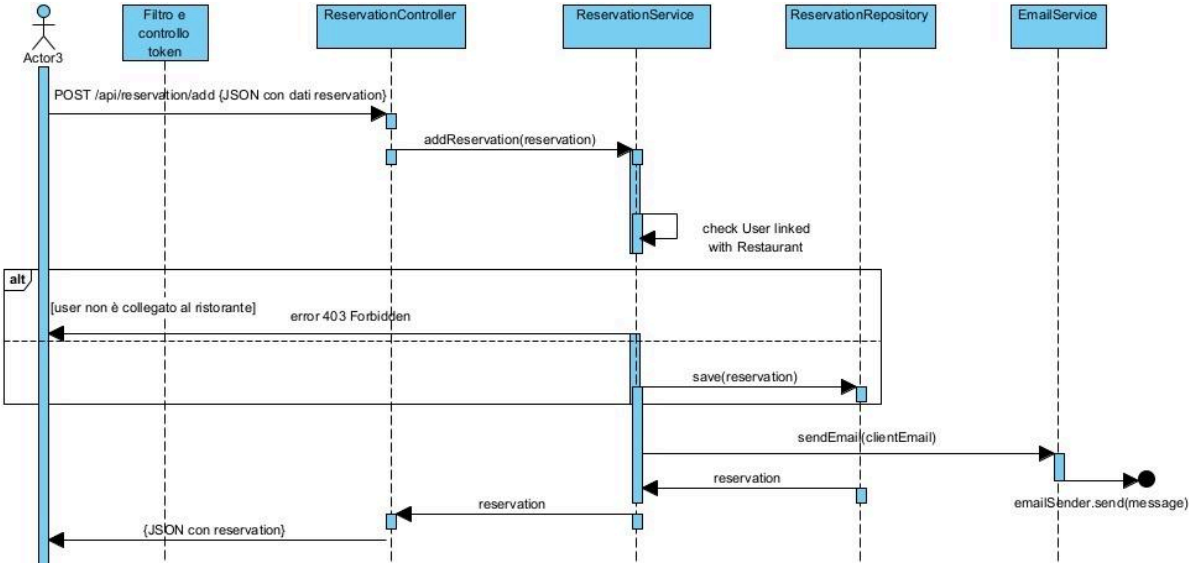


Diagramma di sequenza aggiunta prenotazione

Il processo di aggiunta della prenotazione, avviene con una richiesta *POST* con endpoin /api/reservation/add contenente:

- **JWT** nell'header
- Oggetto **JSON** con i dati della prenotazione (nome, data, numero di persone, email del cliente, id del tavolo al quale apparterrà)

La richiesta viene intercettata dal **JwtAuthenticationFilter** che filtra la richiesta (semplificato nel diagramma).

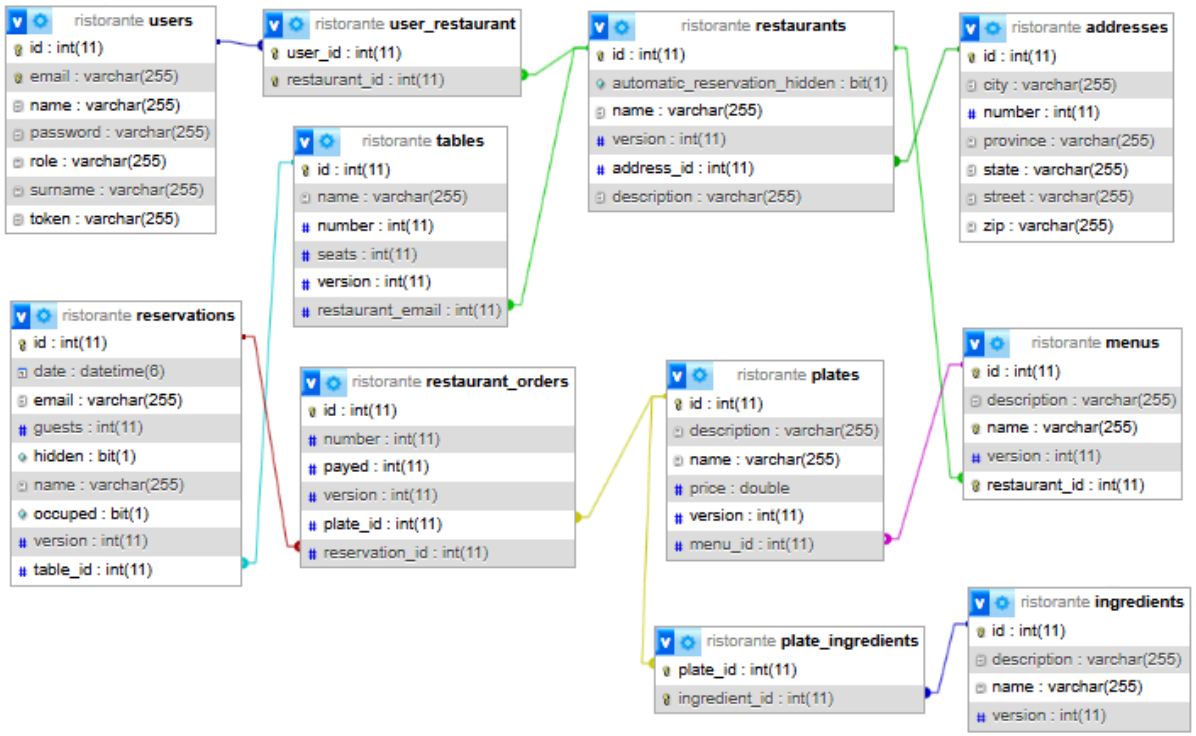
Il **ReservationController** inoltrerà la richiesta a **ReservationService.**

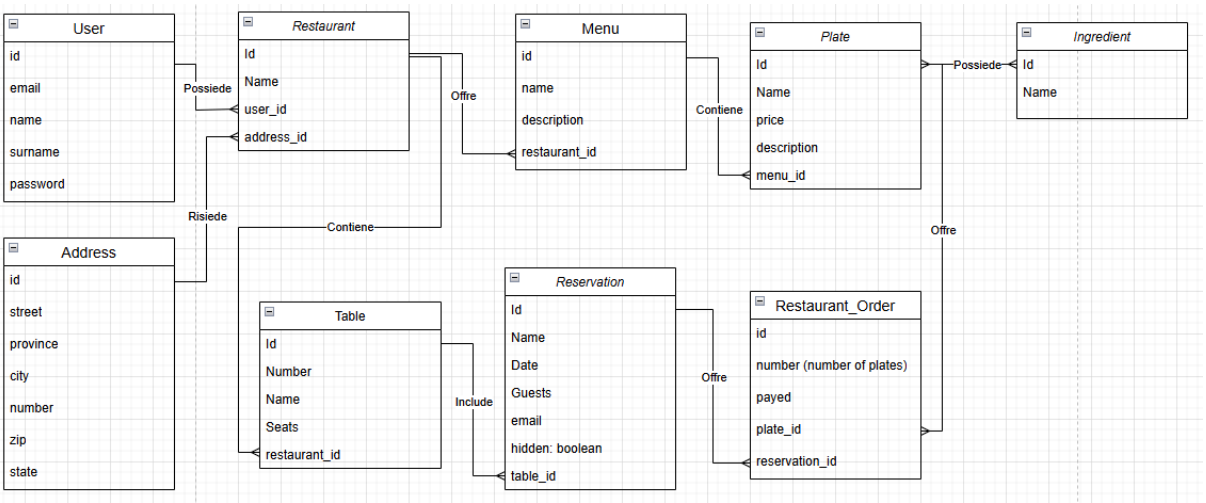
ReservationService, verificata la correttezza di tutti i dati e l'esistenza del tavolo, verificherà che l'utente sia collegato col ristorante. Se è presente la email del cliente, farà richiesta a **EmailService** di inoltrare una mail di conferma prenotazione tramite la libreria **Email** del Framework Spring. Infine salverà il menu tramite il repository.

3.1.2 Database

Tutte le funzionalità sono state implementate sfruttando le capacità di Spring Data JPA, utilizzando specifiche implementazioni delle interfacce Repository, come JpaRepository.

Queste interfacce estendono CrudRepository, offrendo operazioni di CRUD e facilitando il supporto per la gestione dei dati nel database in modo efficiente.


ristorante users
id : int(11)
email : varchar(255)
name : varchar(255)
password : varchar(255)
role : varchar(255)
surname : varchar(255)
token : varchar(255)
ristorante user_restaurant
user_id : int(11)
restaurant_id : int(11)
ristorante restaurants
id : int(11)
automatic_reservation_hidden : bit(1)
name : varchar(255)
version : int(11)
address_id : int(11)
description : varchar(255)
ristorante addresses
id : int(11)
city : varchar(255)
number : int(11)
province : varchar(255)
state : varchar(255)
street : varchar(255)
zip : varchar(255)
ristorante tables
id : int(11)
name : varchar(255)
number : int(11)
seats : int(11)
version : int(11)
restaurant_email : int(11)
ristorante reservations
id : int(11)
date : datetime(6)
email : varchar(255)
guests : int(11)
hidden : bit(1)
name : varchar(255)
occuped : bit(1)
version : int(11)
table_id : int(11)
ristorante restaurant_orders
id : int(11)
number : int(11)
payed : int(11)
version : int(11)
plate_id : int(11)
reservation_id : int(11)
ristorante plates
id : int(11)
description : varchar(255)
name : varchar(255)
price : double
version : int(11)
menu_id : int(11)
ristorante menus
id : int(11)
description : varchar(255)
name : varchar(255)
version : int(11)
restaurant_id : int(11)
ristorante plate_ingredients
plate_id : int(11)
ingredient_id : int(11)
ristorante ingredients
id : int(11)
description : varchar(255)
name : varchar(255)
version : int(11)


User
id
email
name
surname
password
Restaurant
Id
Name
user_id
address_id
Menu
id
name
description
restaurant_id
Plate
Id
Name
price
description
menu_id
Ingredient
Id
Name
Possiede
Offre
Contiene
Possiede
Risiede
Contiene
Offre
Address
id
street
province
city
number
zip
state
Table
Id
Number
Name
Seats
restaurant_id
Include
Reservation
Id
Name
Date
Guests
email
hidden: boolean
table_id
Offre
Restaurant_Order
id
number (number of plates)
payed
plate_id
reservation_id

3.2 FrontEnd

3.2.1 Architettura

L'architettura del frontend, analogamente a quella del backend, segue un modello a strati. Questo approccio favorisce la separazione delle responsabilità, la manutenibilità del codice e la riusabilità dei componenti.

Gli strati principali sono:

- **Service:**
  - Responsabile della comunicazione con il backend
  - Tutte le chiamate HTTP partono da qui
  - Gestione dello stato (utente loggato, ecc.)
  - I service vengono iniettati nei componenti tramite **dependency injection**.

```
@Injectable({
  providedIn: 'root'
})
export class UserService {

  private apiUrl = 'http://localhost:8080/api/users';  // URL del backend

  constructor(private http: HttpClient) { }

  //Mantiene lo stato dell'utente
  private userSubject = new BehaviorSubject<User | null>(null);

  //Espone lo stato come Observable
  user$ = this.userSubject.asObservable();

  getUser(): User | null {
    return this.userSubject.value;
  }

  getUserByEmail(email: string): Observable<User> {
    return this.http.get<User>(`${this.apiUrl}/${email}`);
  }

  //Continua col codice ...
}
```

- **Component**
  - Rappresenta l'interfaccia utente (UI)
  - Include:
    - HTML per la struttura
    - CSS per lo stile
    - TypeScript per la logica
  - Comunica coi service per recuperare o inviare dati

```
@Component({
  selector: 'app-profile',
  templateUrl: './profile.component.html',
  styleUrls: ['./profile.component.scss']
})
export class ProfileComponent implements OnInit {
  constructor(private userService: UserService) {}

  ngOnInit() {
    this.userService.getUserData().subscribe(...);
  }
}
//Continua col codice ...
```

es. Component nella mia applicazione

- L'applicazione è una **Single Page Application (SPA),** grazie al framework di angular.
  Questo significa che:
  Il browser carica una singola pagine HTML iniziale (index.html).
- I contenuti vengono aggiornati dinamicamente tramite il **routing Angular.**
- L'esperienza utente è più veloce e fluida.

```
const routeConfig: Routes = [
  {
    path: ' ',
    component: HomeComponent,
    title: 'Home page',
    children: [
      { path: 'login',
        component: LoginComponent,
        title: 'Login page' },
      { path: 'register',
        component: RegisterComponent,
        title: 'Register page' }
    ]
  },
//Continua col codice ...
```

es. routing nella mia applicazione

3.2.2 Struttura

Il progetto frontend è organizzato seguendo una struttura modulare in cartelle che separano logicamente:

- **components/**: componenti riutilizzabili a livello globale
- **services/:** servizi per chiamate HTTP e gestione dati
- **models/:** interface TypeScript per typing forte
- **assets/:** immagini

Gestione dello stato

Lo stato dell'utente viene gestito nel frontend attraverso una combinazione di **RxJS** e **LocalStorage**, con l'obiettivo di garantire sia **reattività** che **persistenza** dei dati.

RxJS – Stato reattivo condiviso

Per mantenere lo stato utente in memoria e renderlo accessibile da più componenti, è stato utilizzato un oggetto BehaviorSubject di RxJS. Questo tipo di stream consente di conservare l'ultimo valore emesso e notificarlo immediatamente a chi si iscrive.

Nel servizio UserService, il campo:

```
private userSubject = new BehaviorSubject<User | null>(null);
```

mantiene lo stato interno, mentre:

```
user$ = this.userSubject.asObservable();
```

espone quello stato come Observable per permettere ai componenti di sottoscriversi e reagire automaticamente a ogni aggiornamento, senza dover gestire manualmente il flusso dei dati.

LocalStorage – Persistenza locale

Per conservare lo stato anche dopo un aggiornamento della pagina o un riavvio del browser, viene utilizzato il localStorage. Quando lo stato utente viene aggiornato tramite:

```
setUser(user: User | null): void {
  this.userSubject.next(user);
  if (user) {
    localStorage.setItem('user', JSON.stringify(user));
  } else {
    localStorage.removeItem('user');
  }
}
```

i dati vengono serializzati e salvati nel browser. In questo modo, è possibile ripristinarli all'avvio dell'applicazione (es. nel `ngOnInit` del servizio o del componente root), rendendo l'esperienza utente continua e persistente.

Questa architettura consente di combinare:

- la **reattività e l'interoperabilità** dei dati tramite RxJS
- con la **persistenza lato client** garantita da localStorage

Il risultato è una gestione dello stato efficiente, scalabile e facilmente testabile.

JwtInterceptor

Il JwtInterceptor intercetta tutte le richieste HTTP in uscita e, se presente un JWT valido, lo aggiunge automaticamente nell'header Authorization. In questo modo non è più necessario ripetere manualmente la configurazione del token in ogni chiamata API.

Comportamento

1. **Intercettazione**: implementa l'interfaccia HttpInterceptor, intercettando ogni HttpRequest inviata dall'app.

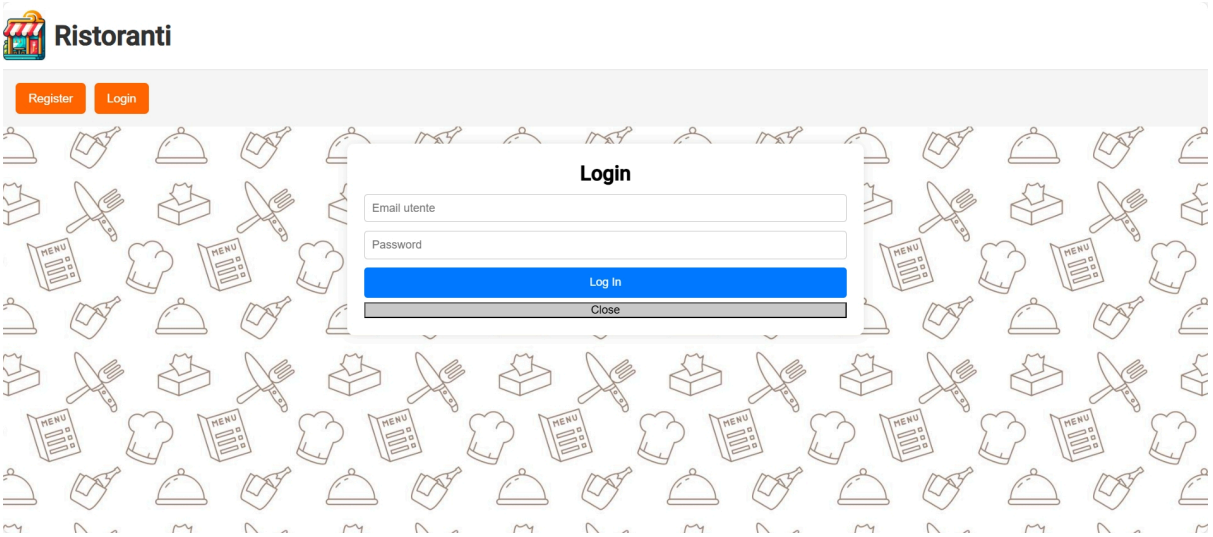
2. **Recupero token**: ottiene il JWT da localStorage
3. **Clonazione della richiesta**: se il token esiste, clona l'HttpRequest aggiungendo l'header:

```
Authorization: Bearer <token>
```

4. **Inoltro**: inoltra la richiesta (modificata o meno) al HttpHandler successivo.

3.2.3 Funzionalità

Prima interazione



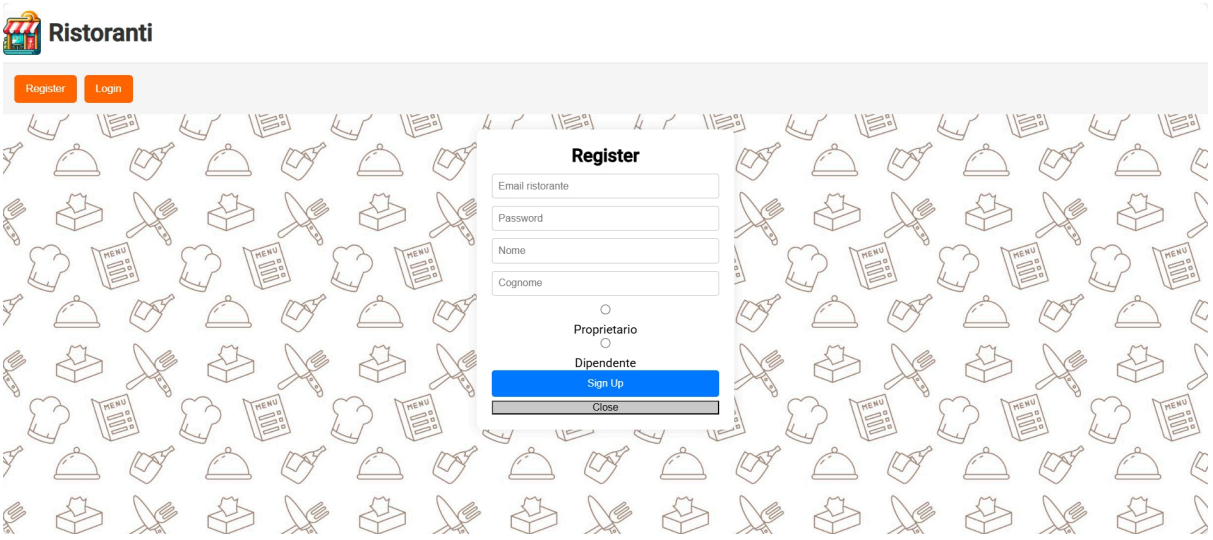
Login percorso: /login

Una volta sulla pagina di login, un utente già registrato può autenticarsi per visualizzare i propri ristoranti. Cliccando sul pulsante "Accedi", email e password vengono passate al servizio ***`user.service.ts`***, che esegue una chiamata POST all'endpoint */api/users/login* con email e password in formato JSON:

```
{
  "email": "utente@example.com",
  "password": "••••••••"
}
```

la richiesta API, se ha successo, avrà come risposta l'utente completo di tutti i dati (esclusa la password), il token JWS generato:

```
{
  "user": {
    "id": "123456",
    "name": "Mario Rossi",
    "surname": "Rossi",
    "email": "utente@example.com",
    "role": "OWNER",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9…"
}
```



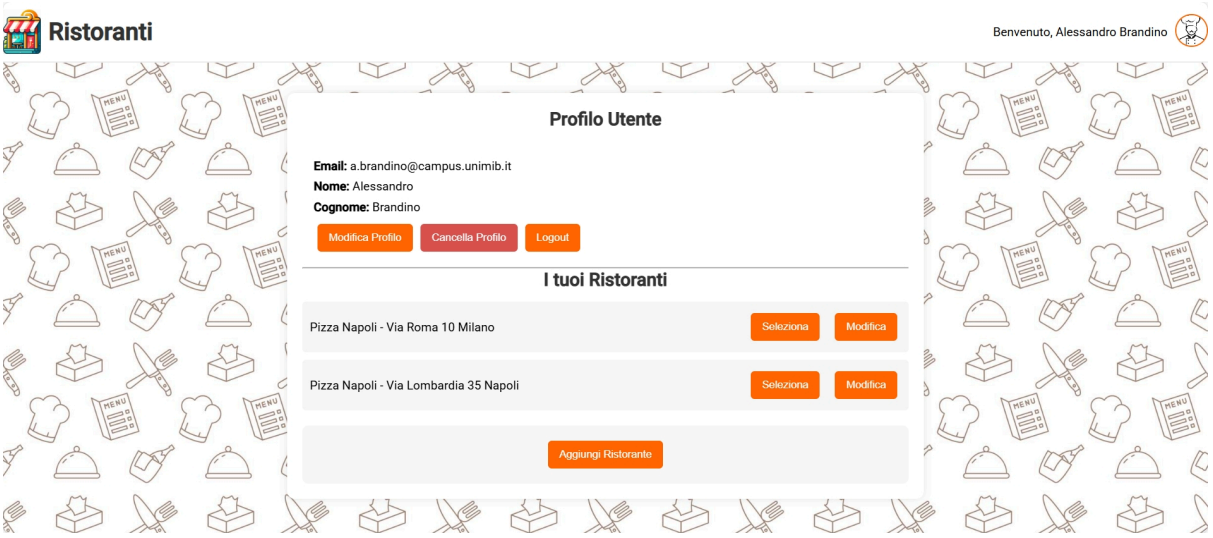
Registrazione percorso: /registrazione

Dalla pagina di registrazione un utente che fa il primo accesso in assoluto può registrarsi creando il proprio profilo utente. Deve compilare tutti i campi e specificare se si tratta di un proprietario di ristoranti o di un dipendente. Cliccando sul pulsante "Sign Up", tutti i dati vengono passati a ***`user.service.ts`***, che esegue una chiamata POST all'endpoint *`/api/users/register`* con i dati in formato JSON:

```
{
  "email": "utente@example.com",
  "password": "••••••••"
  "name": "Matteo",
  "surname": "Bianchi",
```

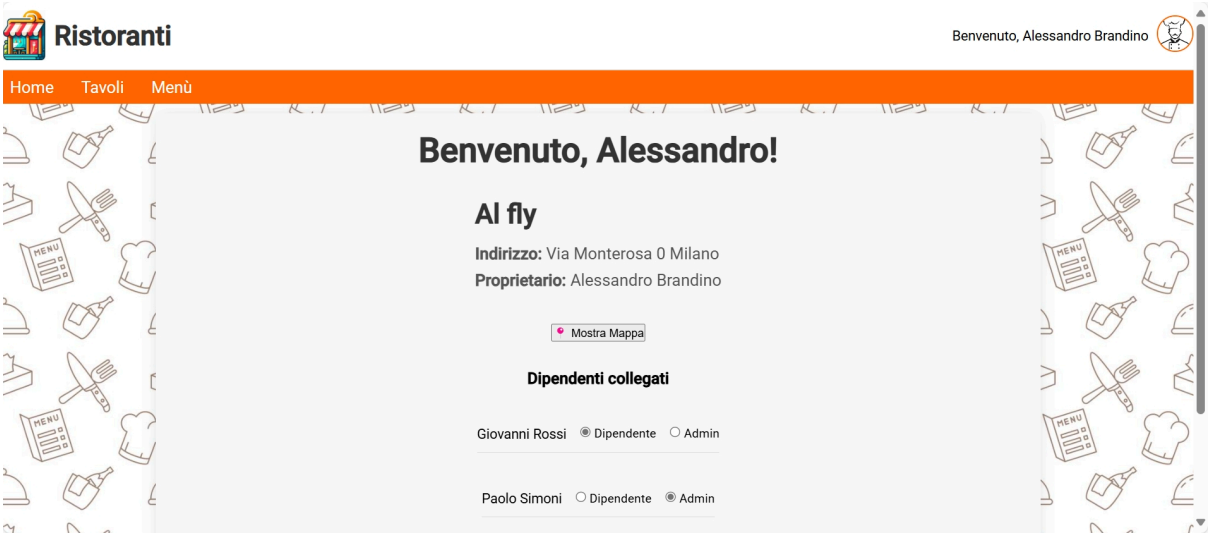
```
  "role": "OWNER", //se dipendente "role": ""
}
```

la richiesta API, se ha successo, avrà una risposta uguale a quella del login, quindi un JSON con tutti i dati dell'utente appena creato

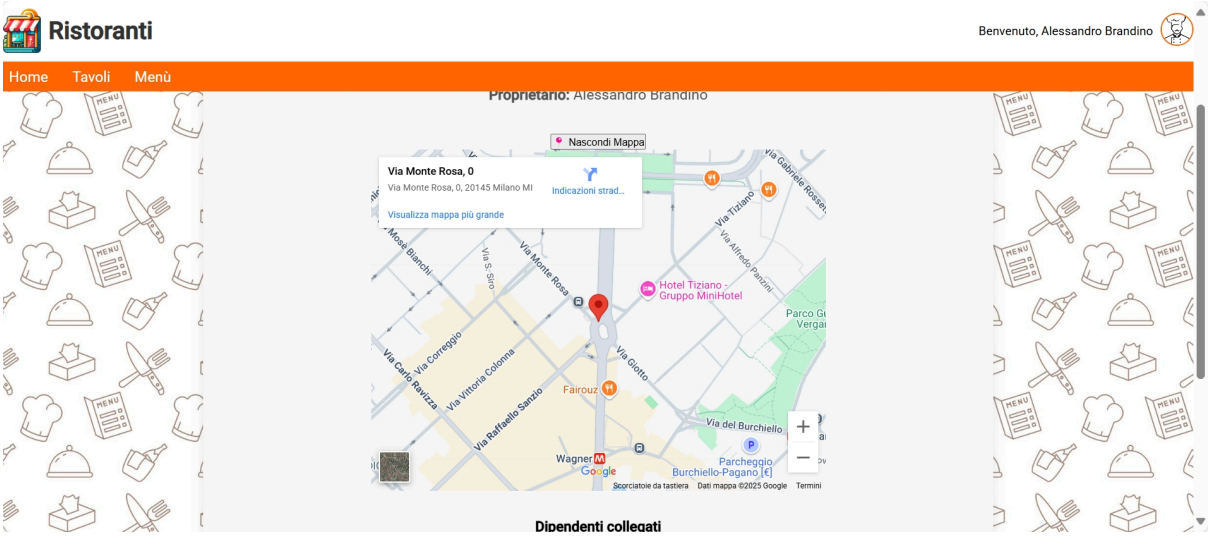


Una volta effettuato l'accesso, la pagina apre il profilo dell'utente nel quale possiamo selezionare il ristorante che vogliamo gestire, modificare il profilo, eliminare il profilo, fare log out, aggiungere un nuovo ristorante.

Gestione del ristorante



Selezionato un ristorante, si aprirà la home del ristorante con le informazioni di esso, quali nome, indirizzo, proprietario e dipendenti. Nel caso in cui l'utente sia il proprietario allora potrà modificare i ruoli dei dipendenti. Inoltre, è possibile visualizzare il punto preciso in cui è il ristorante grazie all'integrazione google maps.



Integrazione Google Maps

Per arricchire l'esperienza utente con la visualizzazione geografica dei ristoranti, è stata integrata l'API JavaScript di Google Maps attraverso il package ufficiale *`@angular/google-maps`*.

1. **Ottenimento Api key**
   - Accedere alla Google Cloud Console.
   - Creare l'API **Maps JavaScript API**.
   - Nella sezione **Credentials**, generare una **API Key**.
2. **Salvataggio chiave nell'environment**

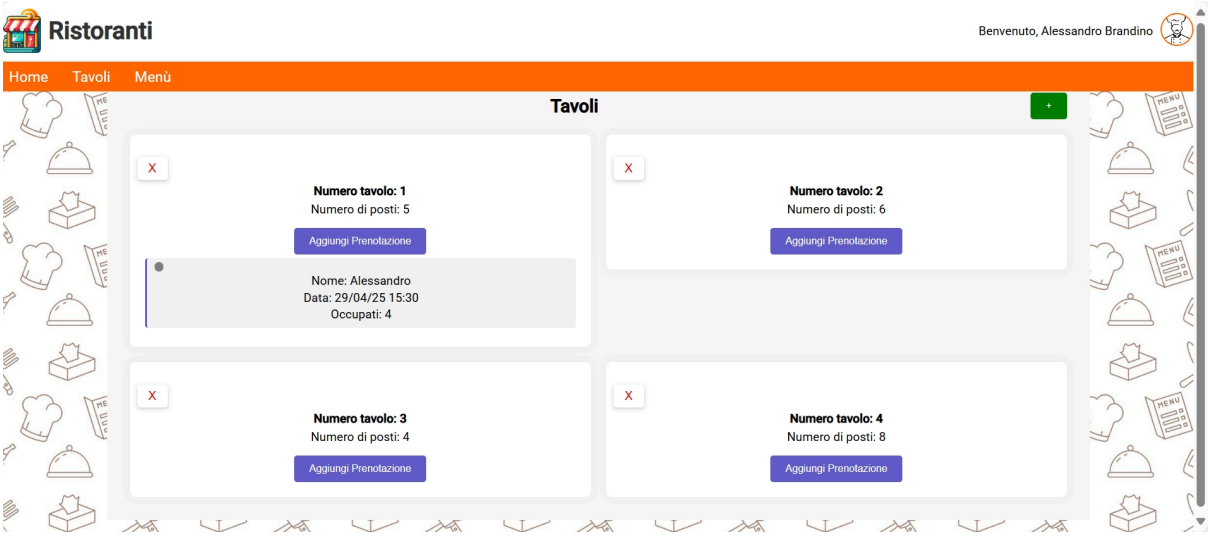
   Aprire *`src/environments/environment.ts`* e aggiungere:

```
export const environment = {
  production: false,
  googleMapsApiKey: 'LA_API_KEY'
};
```

3. **Costruzione dell'url di embedding**

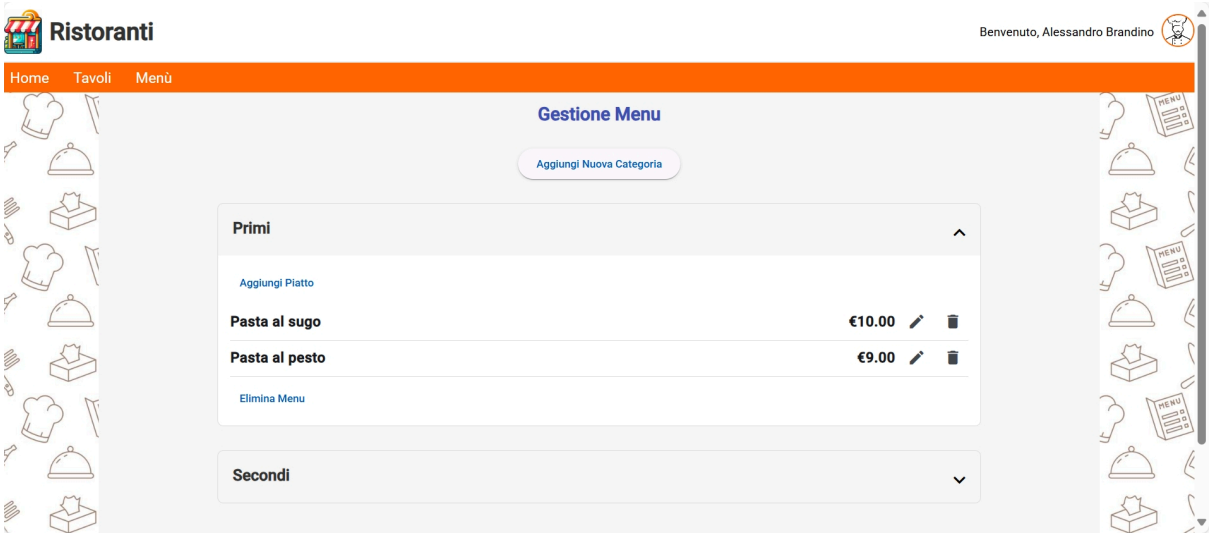
   Nella *main.component.ts,* ovvero il component responsabile del tavolo, avremo un metodo che prende come input il tavolo e restituisce l'URL per l'iFrame

```
private getMapUrl(restaurant: Restaurant): string {
 const address = encodeURIComponent(this.getFullAddress(restaurant));
 const apiKey = environment.googleMapsApiKey;
 return `https://www.google.com/maps/embed/v1/place?key=${apiKey}&q=${address}`;
}
```



Cliccando il bottone *Tavoli* si accede alla pagina nel quale gestire i tavoli del ristorante. Qui si può:

- Aggiungere nuovi tavoli
- Eliminare un tavolo
- Aggiungere una prenotazione
- Accedere alle prenotazioni



4. Test

Sono stati effettuati dei test per verificare la correttezza dell'applicazione.

4.1 Test unitari backend

Il backend utilizza JUnit 5 e Mockito per testare controller e service. Di seguito un esempio esteso per ***`UserController`***.

```
@WebMvcTest(UserController.class)
@AutoConfigureMockMvc
public class UserControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;

    @Test
    void getAllUsers_restituisceListaUtenti() throws Exception {
        List<User> users = List.of(
                new User(1, "Alessandro", "Rossi", "alex@example.com", "OWNER"),
                new User(2, "Luca", "Verdi", "luca@example.com", "ADMIN")
        );
        when(userService.getAllUsers()).thenReturn(users);

        mockMvc.perform(get("/api/users/all"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$[0].name").value("Alessandro"))
                .andExpect(jsonPath("$[1].email").value("luca@example.com"));
    }

    @Test
    void findByEmail_restituisceUtente() throws Exception {
        User user = new User(1, "Test", "User", "test@example.com", "");
        when(userService.findByEmail("test@example.com")).thenReturn(user);

        mockMvc.perform(get("/api/users/test@example.com"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.email").value("test@example.com"));
    }

    @Test
    void login_restituisceUtente() throws Exception {
        User user = new User(1, "Mario", "Rossi", "mario@example.com", "OWNER");
        when(userService.login("mario@example.com", "password")).thenReturn(user);

        String requestBody = "{" +
                "\"email\": \"mario@example.com\"," +
                "\"password\": \"password\"}";

        mockMvc.perform(post("/api/users/login")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(requestBody))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.email").value("mario@example.com"));
    }
```

```
    @Test
    void deleteUser_esistente_restituisceNoContent() throws Exception {
        when(userService.deleteUser(1)).thenReturn(true);

        mockMvc.perform(delete("/api/users/1"))
                .andExpect(status().isNoContent());
    }

    @Test
    void deleteUser_nonEsistente_restituisceNotFound() throws Exception {
        when(userService.deleteUser(99)).thenReturn(false);

        mockMvc.perform(delete("/api/users/99"))
                .andExpect(status().isNotFound());
    }
}
```

4.2 Test umani

I test umani consistono nell'utilizzo diretto dell'applicazione da parte di sviluppatori o utenti di test, al fine di verificare che il sistema funzioni come previsto in condizioni reali. Questo tipo di test è fondamentale per identificare problemi di usabilità, bug non rilevati dai test automatici e comportamenti anomali.

Esempi di scenari testati manualmente:

- **Login e Registrazione**:
  - Inserimento credenziali corrette e verifica del reindirizzamento alla dashboard;
  - Inserimento credenziali errate e visualizzazione messaggio di errore;
  - Registrazione di un nuovo utente e verifica della corretta creazione dell'account.
- **Gestione Ristoranti**:
  - Creazione, modifica ed eliminazione di un ristorante da parte dell'utente OWNER;
  - Controllo della visualizzazione dinamica delle card e della mappa associata.
- **Gestione Tavoli, Menu e Prenotazioni**:
  - Aggiunta, modifica ed eliminazione di tavoli e piatti;
  - Verifica del corretto aggiornamento del frontend dopo ogni operazione;
  - Test di prenotazione: scelta data, tavolo, conferma e annullamento.

Questi test sono stati eseguiti durante la fase di sviluppo sia da me che da alcuni colleghi di Sync Lab. Una volta ultimata è stata testata da utenti veri utenti comuni (non informatici) che ne hanno verificato l'usabilità.

5. Conclusione

Il mio tirocinio in Sync Lab è stata un’esperienza molto interessante, sia dal punto di vista professionale che personale.Questo periodo mi ha dato l’opportunità di approcciarmi a nuove tecnologie, imparare nuovi strumenti e, allo stesso tempo, entrare in contatto con il mondo del lavoro. Ho imparato a gestire la progettazione di un’applicazione da zero, realizzando un progetto basato sulla filosofia del riuso e rispettando le linee guida per scrivere un codice di qualità. In Sync Lab ho trovato persone competenti e disposte a dare un consiglio là dove mi ritrovavo in difficoltà, in un ambiente sereno e accogliente.

5.1 Sviluppi futuri

Gestione inventario

Aggiungere la possibilità di gestire l’inventario permettendo al ristoratore di sapere come sono messi i magazzini del ristorante permettendo di ordinare la merce per tempo.

Applicazione per telefono

Creare un'applicazione analoga mobile così che si possa fruire al ristorante comodamente dai dispositivi mobili.

5.2 Conclusione

In conclusione, sono molto soddisfatto dell’esperienza acquisita durante lo stage. In questo periodo ho sviluppato, oltre a competenze tecniche di progettazione e realizzazione di un'applicazione web, soft skills utili come autogestione e problem solving.
Grazie allo stage in Sync Lab, ho avuto modo di inserirmi in un contesto professionale stimolante, che mi ha permesso di crescere, imparare e confrontarmi con esperti del settore.
Il progetto mi ha anche permesso di trasformare un’idea personale, nata da un’esperienza diretta, in un prodotto concreto e funzionante.

Bibliografia

[1] *Sync Lab, Sync Lab, https://www.synclab.it, Accesso: 8 Aprile 2025*

[2] *Spring, Spring framework, Spring Framework, Accesso 15 Aprile 2025*

[3] *Spring, Spring Data JPA, Spring Data JPA :: Spring Data JPA, Accesso 15 Aprile 2025*

[4] *Spring, Spring Data JPA Repository, Defining Repository Interfaces :: Spring Data JPA, Accesso 15 Aprile 2025*

[5] *Spring, CrudRepository interface, CrudRepository (Spring Data Core 3.4.4 API), Accesso 15 Aprile 2025*

[6] *Spring, ListCrudRepository interface, ListCrudRepository (Spring Data Core 3.4.4 API), Accesso*

[7] *Spring, Spring Security, Spring Security, Accesso 15 Aprile 2025*

[8] Spring, Java JWT, GitHub - jwtk/jjwt: Java JWT: JSON Web Token for Java and Android, Accesso 15 Aprile 2025

[9] Spring, PasswordEncoder, PasswordEncoder :: Spring Security, Accesso 15 Aprile 2025

[10] Spring, Spring Email, Email :: Spring Framework, Accesso 30 Aprile 2025

[11] Angular, Angular, Home • Angular, Accesso 30 Aprile 2025

[12] Mysql, Mysql, MySQL, Accesso 15 Maggio 2025

[13] Intellij, Intellij Idea, IntelliJ IDEA – the IDE for Pro Java and Kotlin Development, Accesso 15 Maggio 2025

[14] Visual Studio Code, Visual Studio Code, Visual Studio Code - Code Editing. Redefined, Accesso 15 Maggio 2025

[15] XAMPP, XAMPP; XAMPP Installers and Downloads for Apache Friends, Accesso 15 Maggio 2025